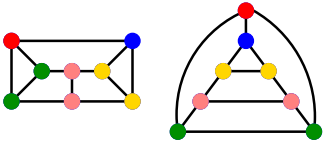
Module 7 project: Graph Isomorphism

Part II: Branching Algorithms



Ruben Hoeksma

What happens now? Group part of the project

Project groups:

- You should be in a project group (on Canvas).
- Your project group should have a team contract by the end of tomorrow.

Deliverables:

- Team contract (tomorrow).
- Sing-off/competition and submission of code (Week 10).
- Submission of report/documentation (Week 10).
- Keep track of performance of (different versions of) your algorithm for the report/documentation.

What happens now? Group part of the project

You have implemented Colour Refinement. Should work with an arbitrary initial colouring (not just uniform).

You will extend those algorithms to solve Graph Isomorphism (GI) and Counting Automorphisms (#Aut), as a group.

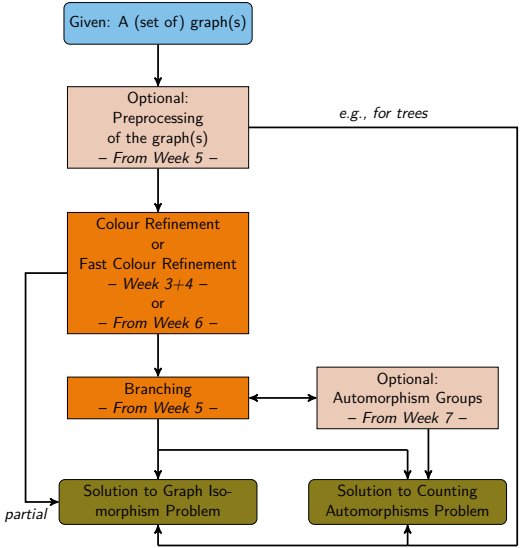
Three more **lectures**:

- Branching (today).
- Fast Colour Refinement.
- Automorphism Groups.

Five **project sessions** to get help with implementation/questions.

Requirements and grading on Canvas (Prj - Home & Prj - Grading Criteria pages).

Overview (also Page 4 of the reader)



Summary last lecture

Recall the *Graph Isomorphism (GI) Problem*, and the *colour refinement* algorithm to *efficiently* compute a *coarsest stable colouring* of a given graph.

Definition

A colouring α of G is stable if for all $u, v \in V(G)$ it holds that:
if $\alpha(u) = \alpha(v)$, then u and v have identically coloured neighborhoods.

Key fact:

Theorem

For every graph G and partition π_0 of $V(G)$, there is a unique coarsest stable partition of $V(G)$ that refines π_0, which is the partition given by colour refinement, when starting with π_0.

Colour Refinement Invariant

A useful invariant that follows from last lecture:

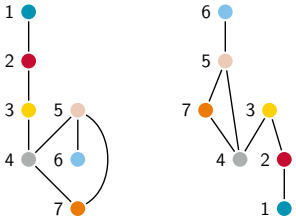
Proposition (Invariant)

Let α be a colouring of a graph set $\mathcal{G} = \{G, H\}$, and let β be the stable colouring of $\mathcal{G}$ given by colour refinement, when starting with α.
Then any isomorphism $f : V(G) \to V(H)$ that is colour preserving for α is also colour preserving for β.

Two particular choices for the *initial colouring* α are the *uniform colouring* or the degree colouring, but today we will also see alternatives.

Colour Refinement Outcome: Example

Let α be the stable colouring of $\mathcal{G} = \{G, H\}$ that results from applying colour refinement, starting with the *uniform colouring*.

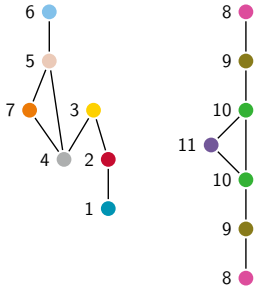


Observation

If α defines bijection $f : V(G) \to V(H)$, then f is unique *isomorphism from G to H.*

Colour Refinement Outcome: Example

Let α be the stable colouring of $\mathcal{G} = \{G, H\}$ that results from applying colour refinement, starting with the *uniform colouring*.

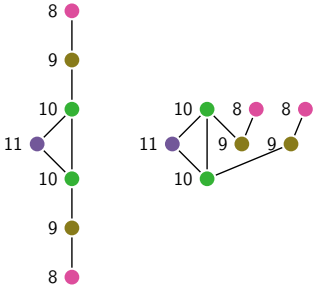


Observation

If α is unbalanced, then there is no isomorphism from G to H.

Colour Refinement Outcome: Example

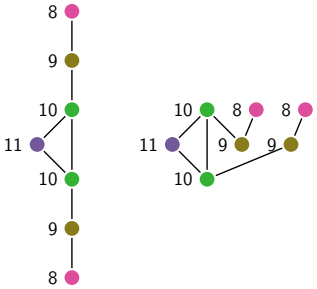
Let α be the stable colouring of $\mathcal{G} = \{G, H\}$ that results from applying colour refinement, starting with the *uniform colouring*.



What if the coarsest stable colouring α is balanced, but does not define a bijection?

Idea: Try all colour preserving bijections f. Here: $2 \cdot 2 \cdot 2 = 8$ possibilities.

Better than Enumeration: Recursive Colour Refinement



Better idea: Choose a vertex x of G for which $f(x)$ is not yet clear.
"Fix" which vertex y of H it should be mapped to. Give x and y a new colour, and apply colour refinement, again.
(Every guess corresponds to a branch of a *recursive algorithm*.)

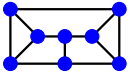
In the example: only two guesses necessary.

Individualisation Refinement - Overview

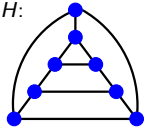
- α: balanced stable colouring of $\mathcal{G} = \{G, H\}$, does not define a bijection.
- Exists colour class C with $2k$ vertices, $k \geq 2$ (k vertices of G and k vertices of H). Denote $C_G = C \cap V(G)$ and $C_H = C \cap V(H)$.
- Choose $x \in C_G$. Every colour preserving isomorphism f has $f(x) = y$ for some $y \in C_H$.
- Try out all such possibilities for $f(x)$ (gives k *branches* of a recursive algorithm): *There exists a colour preserving isomorphism if and only if a colour preserving isomorphism will be found in at least one branch.*
- In each branch (k many), the *choice* $f(x) = y$ is encoded by giving both vertices a *new, unique colour.*
- Given this start colouring, we can apply colour refinement again, and continue recursively until either an isomorphism is found, or it is concluded that no isomorphism with $f(x) = y$ exists.

Example

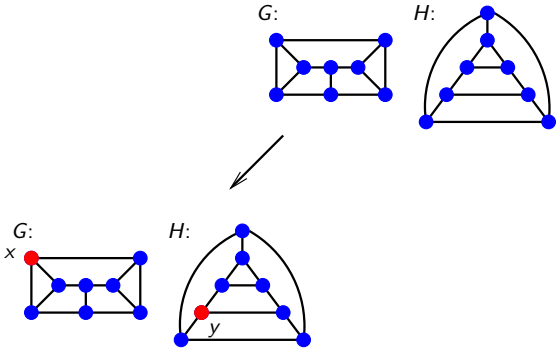
G:



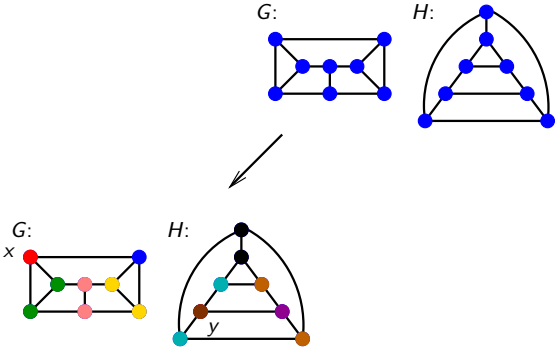
H:



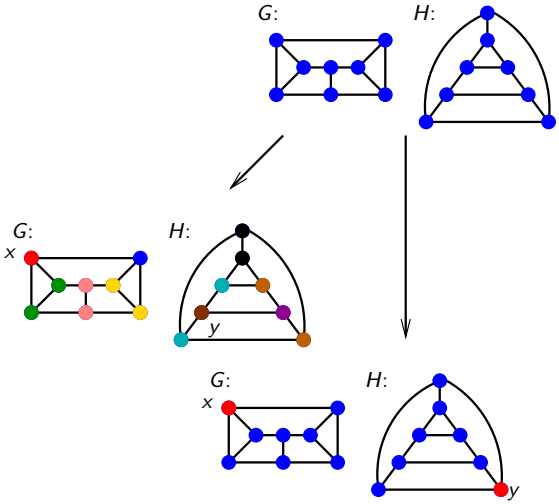
Example



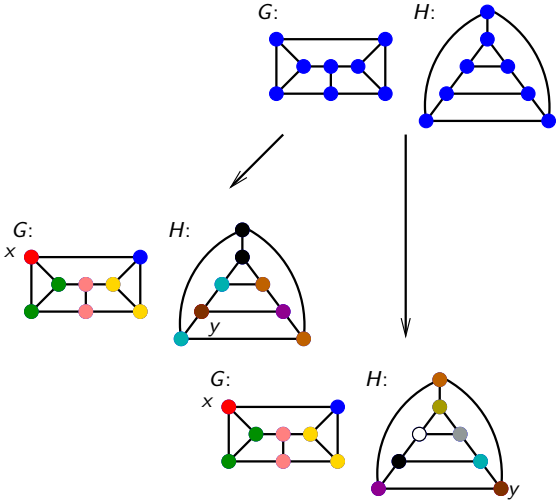
Example



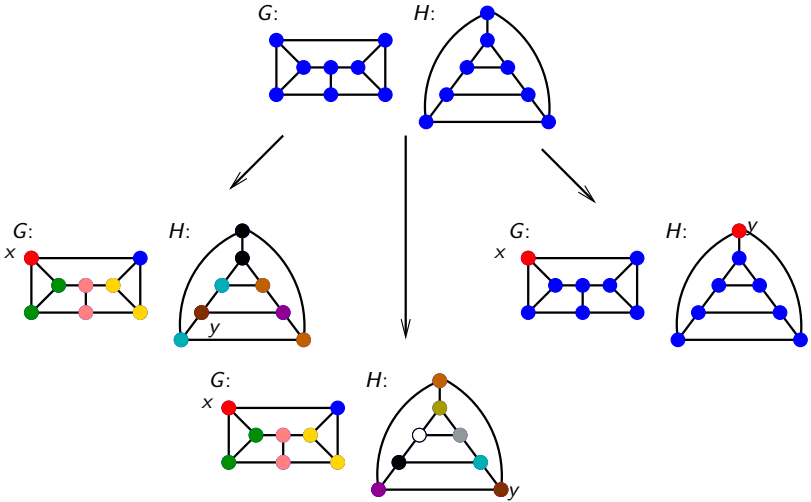
Example



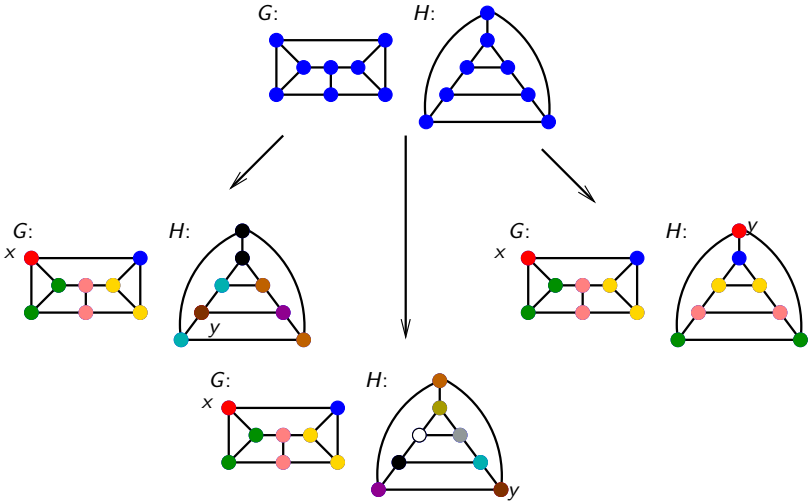
Example



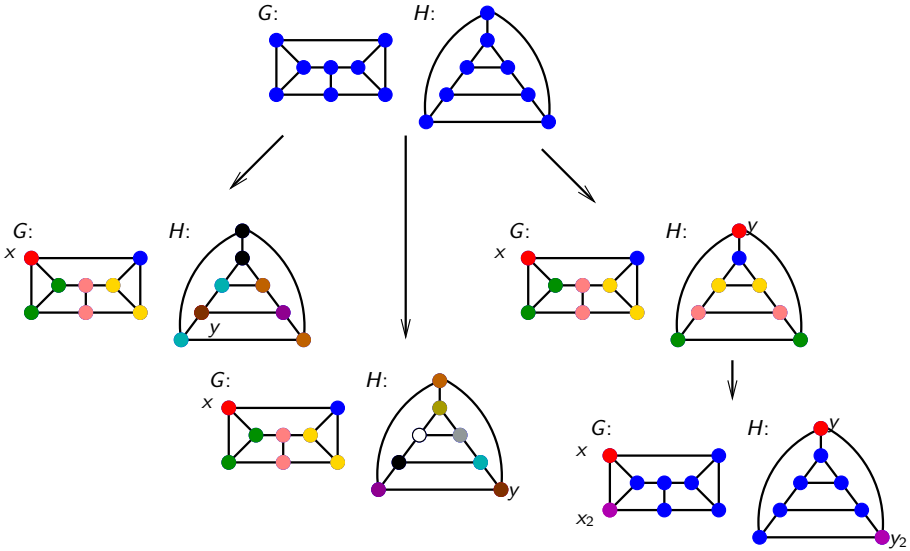
Example



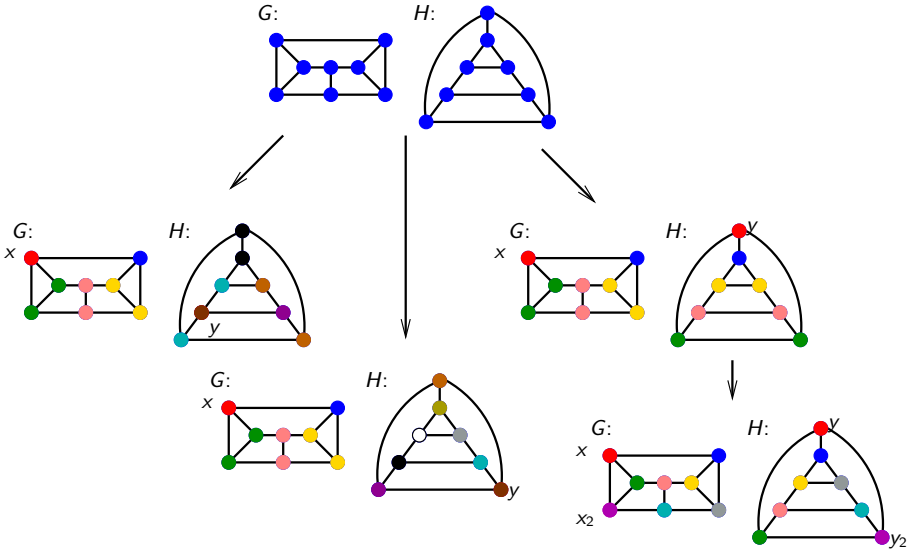
Example



Example



Example



Individualisation Refinement - Implementation Details

- Let $D = (x_1, \ldots, x_d)$ and $I = (y_1, \ldots, y_d)$ be sequences of distinct vertices of G and H, respectively.
- A bijection $f : V(G) \to V(H)$ *follows* D, I if for all $i \in \{1, \ldots, d\}$: $f(x_i) = y_i$.
- Let $\alpha(D, I)$ be the colouring of $\mathcal{G} = \{G, H\}$ that assigns colour i to x_i and y_i for $i \in \{1, \ldots, d\}$, and colour 0 to all vertices not in $D \cup I$.

Proposition

Let β be the coarsest stable colouring of $\mathcal{G} = \{G, H\}$ that refines $\alpha(D, I)$. Every isomorphism f from G to H that follows D, I is β-color preserving.

Corollary

If β unbalanced, $\nexists$ isomorphism from G to H following D, I. If β defines bijection f, f is the unique *isomorphism from G to H following D, I.*

Algorithm 2: Individualisation Refinement

Subroutine $\textsc{CountIsomorphism}(D, I)$:
$\textsc{Input}$: D and I are equal length sequences of vertices of G resp. H.
$\textsc{Output}$: The number of isomorphisms from G to H that follow D, I.

Compute the coarsest stable colouring β of $\mathcal{G} = \{G, H\}$ that refines $\alpha(D, I)$
if β is unbalanced:
    return 0
if β defines a bijection:
    return 1
Choose a colour class C with $|C| \geq 4$.
Choose $x \in C \cap V(G)$.
num $= 0$
for all $y \in C \cap V(H)$:
    num := num+$\textsc{CountIsomorphism}(D + x, I + y)$
return num

Correctness of Individualisation Refinement

Theorem

Algorithm 2 is correct; it returns the number of isomorphisms from G to H that follow D, I.

Proof Sketch: By induction over the recursion tree:
Base: the unbalanced / bijection case (use the corollary).
Induction step: use the previous proposition. □

Theorem

Algorithm 2 terminates.

Proof Sketch: In every recursive call D and I grow by one element.
If $|D| = |I| = |V(G)| = |V(H)|$, the algorithm is done. Thus, the recursion depth is at most $|V(G)|$ and the algorithm terminates. □

Summary of Individualisation Refinement

- Algorithm 2 computes the number of isomorphisms between two graphs G and H. (When called with D and I both empty.)
- The number of recursive calls is at least the number of isomorphisms between G and H, which can be exponential. (worst case: $n!$)
- Implementation for GI (decision problem): terminate as soon as one isomorphism is found!
- By choosing $H := G$, we can solve the #Aut problem (counting the number of automorphisms of G).

Project: This Week

- Implement in Python an individualisation refinement algorithm, that can correctly solve the GI and the #Aut Problem.
- This requires the *colour refinement* algorithm to work with any initial colouring - consider the colour of the vertex (not only the colours of its neighbours).
- *Instances* to test algorithms for GI and #Aut problems, available on Canvas.
- Use small instances (for verifying correctness), and large instances (for tuning the performance).

Milestone:

- End of this week, Python program that can correctly solve the GI problem and #Aut problem for "small" instances.

- Along the way: *improving the computational complexity* is useful (for getting *bonus points* at the end). Line profiler plugin for PyCharm can be useful.
- Additional: Branching rules and preprocessing of graphs (see reader).

Branching Rules

Improvements are possible by choosing the colour class C for branching cleverly.

Implementation idea: Try out and compare different *branching rules*.
Find at least one rule that performs better than straightforward rules, and support this with extensive computational experiments. (See Project Grading Criteria on Canvas)

About tracking performance

- For every algorithmic improvement that you implement, you need to document what the performance change is.
- If the change in performance is not significant, you are probably not done yet!
- Significant is usually: orders of magnitude faster for (at least) certain instances, without being orders of magnitude slower on others.
- It can happen that certain improvements "overshadow" others. In that case, provide performance change of each separately.
- Be complete! E.g., do not only document performance on small instances.

Example performance documentation:

instance	instance size $\lvert V \rvert$	# autom.	comp. times (s)	
			standard	improvement1
example1	35	100	200s	199s
example2	15	20	10s	12s
example3	200	16	>600s	100s
example4	46	1	300s	300s